

Reviewer Pack — Minimal Rank of Quasi-Monte Carlo Sequences Bounds Tseitin C...

Entry 1cefd11c442a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 15:48:08 UTC

Minimal Rank of Quasi-Monte Carlo Sequences Bounds Tseitin Circuit Size

Entry ID: 1cefd11c442a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 15:48:08 UTC

1. Conjecture

Field A (mathematical branch): Quasi-Monte Carlo Theory **Field B** (complexity object): Complexity Theory: Tseitin Circuit Complexity

Statement:

[“For every Boolean function f represented by a Tseitin circuit with n variables and depth D , the minimal distance between any two distinct points in the quasi-Monte Carlo sequence (QMC) of degree D bounds the size of f ’s Tseitin circuit, i.e., $E[\min_dist(QMC)] = \Theta(2^{(D/2)})$.”, ‘For all Tseitin circuits of size S and depth D , there exists a quasi-Monte Carlo sequence of degree D with minimal distance at least $2^{(S/D)}$.’]

Rationale (proposer’s reasoning):

[‘Quasi-Monte Carlo sequences are known for their excellent equidistribution properties, which might provide a stronger control over circuit complexity.’, ‘This conjecture suggests that the regularity of QMC sequences could be leveraged to construct circuits with lower depth.’]

Taxonomy category: TSEITIN_CIRCUITS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9623dd53cb6c85dd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient (ρ) between the computed minimal distance in the quasi-Monte Carlo sequence and the Tseitin circuit size is $\rho \geq 0.7$, across at least 30 independent seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_tseitin_circuit(size, depth):
    if size == 1:
        return ['x0']
    elif depth == 1:
        return [f'NOT {generate_tseitin_circuit(1, 1)[0]}']

    left = generate_tseitin_circuit(size // 2, depth - 1)
    right = generate_tseitin_circuit(size - size // 2, depth - 1)
    var = f'x{size}'
    return [f'{var}', f'AND {left[0]} {right[0]}', f'OR {left[0]} {not_var(right[0])}']

def not_var(var):
    if var.startswith('NOT '):
        return var[4:]
    else:
        return f'NOT {var}'

def generate_qmc_sequence(circuit, degree):
    n = len(circuit)
    qmc = []
    for _ in range(2**degree):
        point = [random.choice([0, 1]) for _ in range(n)]
        if all(point[i] == (point[0] ^ point[1] ^ point[2]) % 2 for i in range(3)):
            qmc.append(point)
    return qmc

def compute_min_dist(qmc):
    n = len(qmc[0])
    min_dist = float('inf')
    for i in range(len(qmc)):
        for j in range(i + 1, len(qmc)):
            dist = sum(abs(qmc[i][k] - qmc[j][k]) for k in range(n))
            if dist < min_dist:
```

```

        min_dist = dist
    return min_dist

def run_trial(seed: int) -> dict:
    random.seed(seed)

    size = random.randint(1, 40)
    depth = random.randint(1, 10)
    circuit = generate_tseitin_circuit(size, depth)
    qmc = generate_qmc_sequence(circuit, depth)
    min_dist = compute_min_dist(qmc)

    return {
        "metric_name": "min_dist",
        "metric_value": min_dist,
        "instances_tested": len(qmc),
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_d = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_d} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_d} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=' ' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'min_dist', 'metric_value': 0, 'instances_tested': 124, 'conjecture_holds': False}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_035ba032.py", line 76, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_035ba032.py", line 61, in run_trial
    min_dist = compute_min_dist(qmc)
               ^^^^^^^^^^^^^^^^^^^

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_035ba032.py", line 45, in compute_min_d
    n = len(qmc[0])
           ~~~^^^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to ensure it can run to completion.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11008
2	preregistration	ollama_remote	glm4:latest	0	0	5636
3	novelty	ollama_remote	glm4:latest	0	0	4761
4	novelty	ollama_remote	glm4:latest	0	0	5145
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16618
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13313
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10474
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8986
9	judge	ollama_remote	glm4:latest	0	0	8753

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 84695 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/1cefd11c442a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1cefd11c442a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1cefd11c442a.tar.gz* (if generated)